CSA4704 – Deep Learning

Experiments

1,Vector and Matrix Operations

Aim

To explore a dataset using descriptive statistics, identify missing values, calculate covariance and correlation matrices, and visualize correlations using a heatmap.

Formulas

1. Dot Product

Dot Product = $(a_1 \times b_1) + (a_2 \times b_2) + \ldots + (a_n \times b_n)$

2. Matrix Multiplication

$C = A \times B$

3. Matrix Transpose

A^T = Transpose of A

4. Matrix Inverse

$A \times A^{-1} = I$

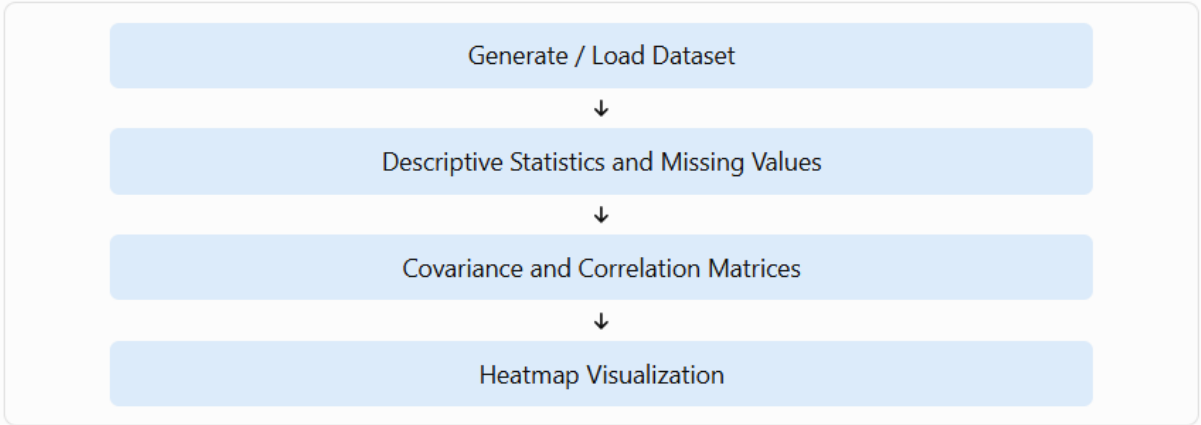
5. Cosine Similarity

Cosine Similarity = $(A \cdot B) / (\|A\| \times \|B\|)$

6. Dense Layer

$y = W \times x + b$

Architecture diagram



Python code

```
import numpy as np
import matplotlib.pyplot as plt
C = np.array([
   [5, 4, 1, 2, 3],
   [4, 5, 1, 2, 3],
   [1, 2, 5, 4, 1],
   [2, 1, 4, 5, 2],
   [5, 3, 2, 1, 4],
   [1, 1, 4, 5, 3]
], dtype=float)
B = np.array([
   [0.8, 0.1, 0.2, 0.5],
   [0.7, 0.2, 0.1, 0.4],
   [0.1, 0.8, 0.7, 0.2],
   [0.2, 0.7, 0.8, 0.1],
   [0.4, 0.2, 0.3, 0.9]
])
print("Dot product:", np.dot(C[0], C[1]))
scores = C @ B
print("\nCustomer-book score matrix:\n", scores)
print("\nTranspose:\n", C.T)
A = np.array([[2, 1], [1, 2]], dtype=float)
print("\nInverse:\n", np.linalg.inv(A))
print("\nAddition:\n", C + C)
print("\nMultiplication:\n", C * C)
norms = np.linalg.norm(C, axis=1)
similarity = (C @ C.T) / np.outer(norms, norms)
print("\nCosine similarity:\n", np.round(similarity, 3))
np.fill_diagonal(similarity, -1)
i, j = np.unravel_index(np.argmax(similarity), similarity.shape)
```

```
print(f"\nMost similar customers: {i+1} and {j+1}")
print("Similarity:", round(similarity[i, j], 3))
x = C[0]
W = B.T
bias = np.array([0.5, 0.2, 0.1, 0.3])
y = W @ x + bias
print("\nDense layer output:", y)
print("Matrix score before bias:", scores[0])
print("Score plus bias:", scores[0] + bias)
plt.imshow(scores, cmap="YlGnBu", aspect="auto")
plt.colorbar(label="Recommendation Score")
plt.xlabel("Books")
plt.ylabel("Customers")
plt.xticks(range(4), ["Book 1", "Book 2", "Book 3", "Book 4"])
plt.yticks(range(6), [f"C{i}" for i in range(1, 7)])
plt.title("Customer-Book Score Matrix")
plt.show()
```

Output

```
Dot product: 54.0

Customer-book score matrix:
 [[8.5 4.1 4.6 7.2]
 [8.4 4.2 4.5 7.1]
 [3.9 7.5 7.4 3.6]
 [4.5 7.5 7.9 4.5]
 [8.1 4.2 4.7 7.8]
 [4.1 7.6 8.  4.9]]

Transpose:
 [[5. 4. 1. 2. 5. 1.]
 [4. 5. 2. 1. 3. 1.]
 [1. 1. 5. 4. 2. 4.]
 [2. 2. 4. 5. 1. 5.]
 [3. 3. 1. 2. 4. 3.]]

Inverse:
 [[ 0.66666667 -0.33333333]
 [-0.33333333  0.66666667]]

Addition:
 [[10.  8.  2.  4.  6.]
 [ 8. 10.  2.  4.  6.]
```

```
 [ 2.  4. 10.  8.  2.]
 [ 4.  2.  8. 10.  4.]
 [10.  6.  4.  2.  8.]
 [ 2.  2.  8. 10.  6.]]

Multiplication:
 [[25. 16.  1.  4.  9.]
 [16. 25.  1.  4.  9.]
 [ 1.  4. 25. 16.  1.]
 [ 4.  1. 16. 25.  4.]
 [25.  9.  4.  1. 16.]
 [ 1.  1. 16. 25.  9.]]

Cosine similarity:
 [[1.    0.982 0.57  0.648 0.964 0.598]
 [0.982 1.    0.59  0.629 0.927 0.598]
 [0.57  0.59  1.    0.949 0.57  0.93 ]
 [0.648 0.629 0.949 1.    0.648 0.981]
 [0.964 0.927 0.57  0.648 1.    0.617]
 [0.598 0.598 0.93  0.981 0.617 1.   ]]

Most similar customers: 1 and 2
Similarity: 0.982

Dense layer output: [9.  4.3 4.7 7.5]
Matrix score before bias: [8.5 4.1 4.6 7.2]
   Score plus bias: [9.  4.3 4.7 7.5]
```



Experiment 2: Covariance, Eigenvalues and Eigenvectors

Aim

To calculate eigenvalues and eigenvectors of a covariance matrix, sort them by eigenvalue, reduce the dataset to two dimensions, visualize principal directions, and calculate the percentage of variance captured by each component.

Formulas

1. Mean

Mean = Sum of all values / Number of values

2. Standardisation (Z-score)

$Z = (X - \mu) / \sigma$

3. Covariance

$Cov(X, Y) = \Sigma[(Xi - \bar{X}) \times (Yi - \bar{Y})] / (n - 1)$

4. Correlation Coefficient

$r = Cov(X, Y) / (\sigma X \times \sigma Y)$

5. Eigenvalue Equation

$|A - \lambda I| = 0$

6. Eigenvector Equation

$A \times v = \lambda \times v$

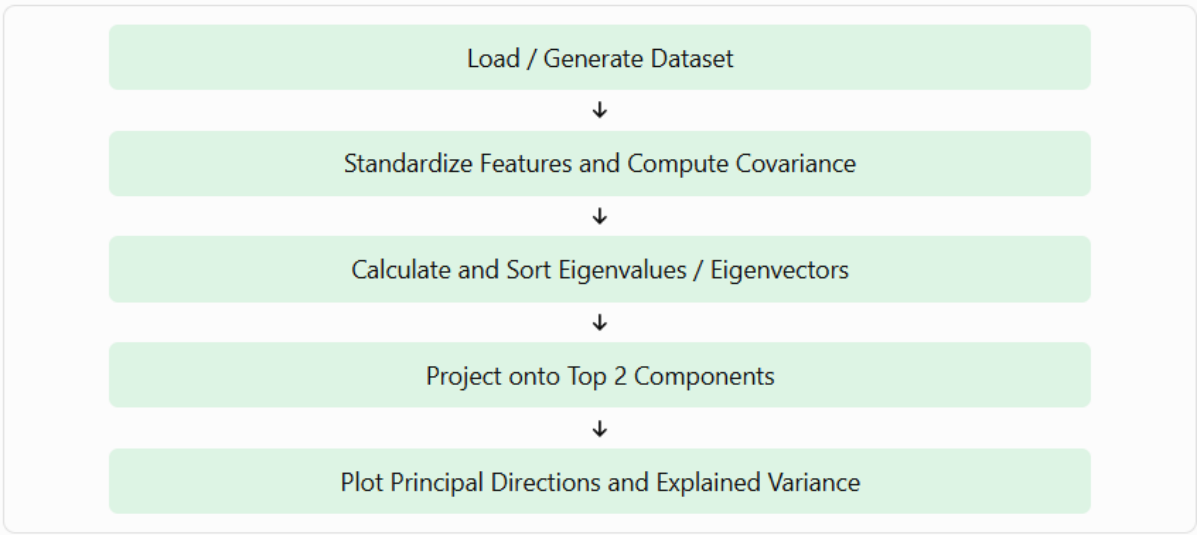
7. Explained Variance Ratio

EVR = (Individual Eigenvalue / Sum of All Eigenvalues) × 100

8. Cumulative Explained Variance

CEV = (Sum of Top k Eigenvalues / Sum of All Eigenvalues) × 100

Architecture diagram



Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
n = 200
hb = np.random.normal(13, 1.5, n)
rbc = 4.5 + 0.25 * (hb - 13) + np.random.normal(0, 0.3, n)
glucose = np.random.normal(100, 15, n)
cholesterol = 180 + 0.5 * (glucose - 100) + np.random.normal(0, 12, n)
bmi = np.random.normal(25, 4, n)
df = pd.DataFrame({
   "Haemoglobin": hb,
   "RBC": rbc,
   "Glucose": glucose,
   "Cholesterol": cholesterol,
   "BMI": bmi
})
X = (df - df.mean()) / df.std(ddof=0)
print("Standardized data:\n", X.head())
cov = X.cov()
corr = X.corr()
print("\nCovariance matrix:\n", cov.round(3))
print("\nCorrelation matrix:\n", corr.round(3))
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, matrix, title in [
   (axes[0], cov, "Covariance"),
   (axes[1], corr, "Correlation")
]:
   im = ax.imshow(matrix, cmap="coolwarm", vmin=-1, vmax=1)
   ax.set_xticks(range(5), df.columns, rotation=45, ha="right")
```

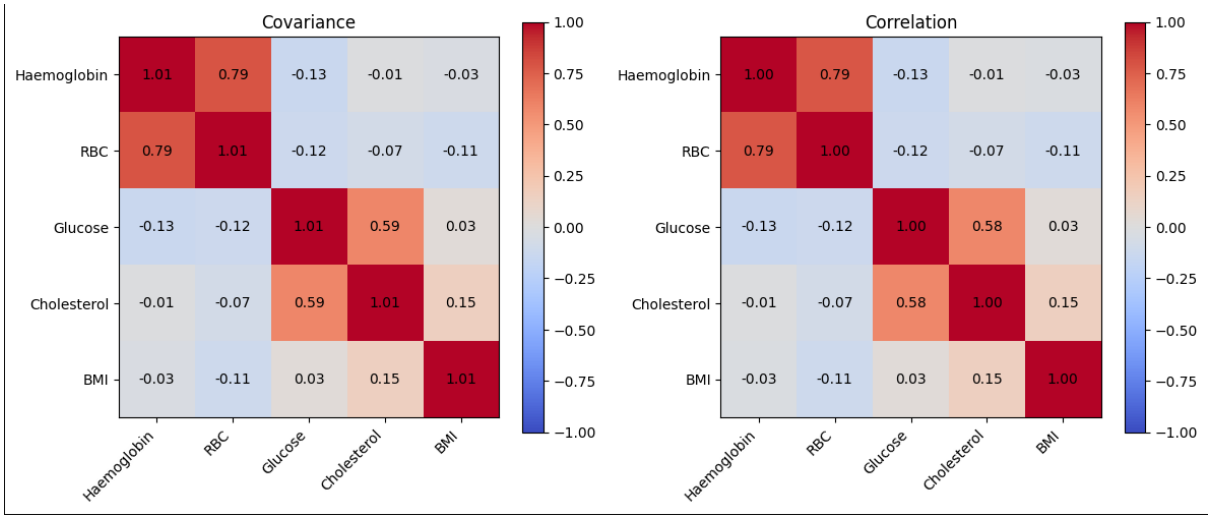
```
        ax.set_yticks(range(5), df.columns)
        ax.set_title(title)
        for i in range(5):
            for j in range(5):
                ax.text(j, i, f"{matrix.iloc[i, j]:.2f}",
                        ha="center", va="center")
        fig.colorbar(im, ax=ax)
    plt.tight_layout()
    plt.show()
    eigenvalues, eigenvectors = np.linalg.eigh(cov)
    idx = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]
    variance = eigenvalues / eigenvalues.sum() * 100
    print("\nSorted eigenvalues:\n", eigenvalues)
    print("\nSorted eigenvectors:\n", eigenvectors)
    print("\nVariance captured (%):\n", variance.round(2))
    projected = X.to_numpy() @ eigenvectors[:, :2]
    plt.scatter(projected[:, 0], projected[:, 1], alpha=0.6)
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("Patient Data in 2-D PCA Space")
    plt.arrow(0, 0, 2, 0, head_width=0.15, color="red")
    plt.arrow(0, 0, 0, 2, head_width=0.15, color="green")
    plt.text(2, 0, "PC1")
    plt.text(0, 2, "PC2")
    plt.grid()
    plt.show()
```

Output

```
Covariance matrix:
              Haemoglobin    RBC  Glucose  Cholesterol    BMI
Haemoglobin         1.005  0.792   -0.135       -0.013 -0.031
```

```
RBC                0.792  1.005   -0.119       -0.071 -0.111
Glucose           -0.135 -0.119    1.005        0.587  0.026
Cholesterol       -0.013 -0.071    0.587        1.005  0.153
BMI               -0.031 -0.111    0.026        0.153  1.005

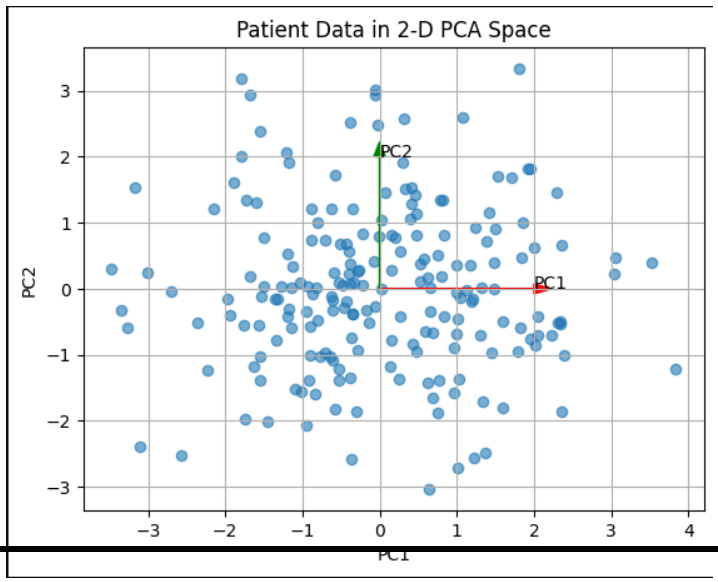
Correlation matrix:
              Haemoglobin    RBC  Glucose  Cholesterol    BMI
Haemoglobin         1.000  0.788   -0.134       -0.013 -0.031
RBC                 0.788  1.000   -0.118       -0.071 -0.111
Glucose            -0.134 -0.118    1.000        0.584  0.026
Cholesterol        -0.013 -0.071    0.584        1.000  0.153
   BMI             -0.031 -0.111    0.026        0.153  1.000
```




```
Sorted eigenvalues:
 [1.91954254 1.50661358 0.99319498 0.40151703 0.20425749]

Sorted eigenvectors:
 [[-0.59321072  0.38130196 -0.09634703 -0.04963721 -0.70068771]
 [-0.60817947  0.35256776  0.00613654  0.14928795  0.69533385]
 [ 0.38116254  0.56114977  0.24992113  0.68362343 -0.10012237]
 [ 0.32760375  0.6331292   0.01670924 -0.69178914  0.11389374]
 [ 0.16003025  0.12067784 -0.96329633  0.1712777   0.05051042]]

Variance captured (%):
    [38.2  29.98 19.76  7.99  4.06]
```



Experiment 3: Probability Distributions and Bayes' Theorem

Aim

To generate and visualize Bernoulli, Binomial, Poisson, Uniform and Gaussian distributions, calculate the probability of disease after a positive test using Bayes' theorem, verify it through simulation, and study the effect of disease prevalence.

Formulas

1. Bernoulli Distribution

$P(X = x) = p^x \times (1 - p)^{(1 - x)}$

2. Binomial Distribution

$P(X = k) = {}^nC_k \times p^k \times (1 - p)^{n-k}$

3. Poisson Distribution

$P(X = k) = (e^{-}\lambda \times \lambda^k) / k!$

4. Uniform Distribution

$f(x) = 1 / (b - a)$

5. Gaussian (Normal) Distribution

$f(x) = [1 / (\sigma \times \sqrt{(2\pi)})] \times e^{[-(x - \mu)^2 / (2\sigma^2)]}$

6. Bayes' Theorem

$P(A \mid B) = [P(B \mid A) \times P(A)] / P(B)$

7. Bayes' Theorem for Disease Detection

P(Disease | Positive) =

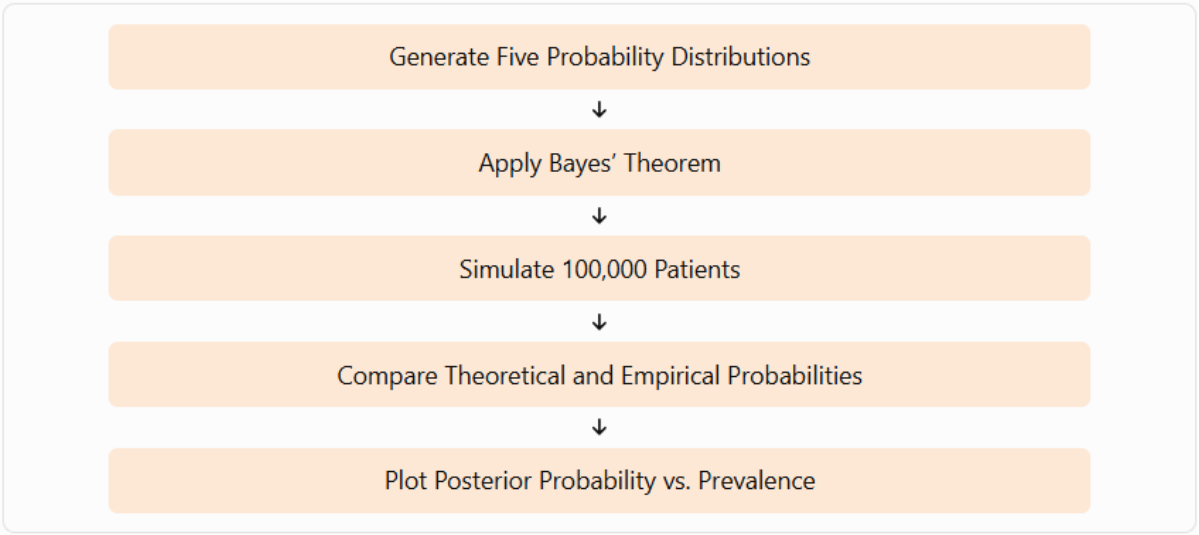
[Sensitivity × Prevalence] /

[(Sensitivity × Prevalence) + (1 − Specificity) × (1 − Prevalence)]

8. Empirical Probability

P(A) = Number of Favourable Outcomes / Total Number of Outcomes

Architecture diagram



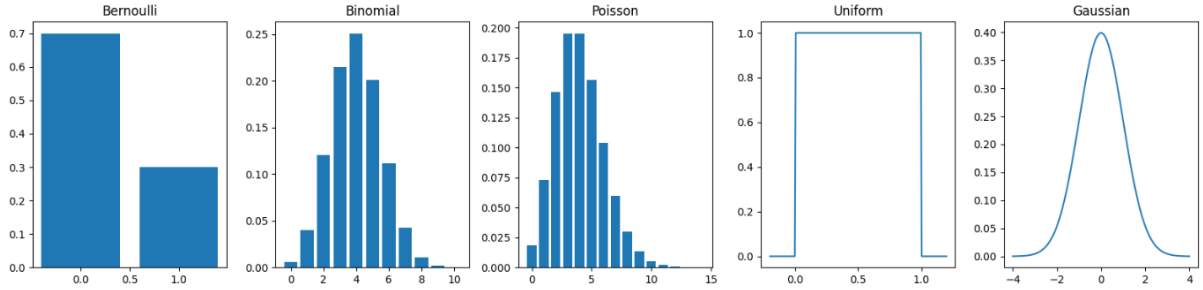
Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import bernoulli, binom, poisson, uniform, norm
np.random.seed(42)
fig, axes = plt.subplots(1, 5, figsize=(16, 4))
x = np.arange(2)
axes[0].bar(x, bernoulli.pmf(x, 0.3))
axes[0].set_title("Bernoulli")
x = np.arange(11)
axes[1].bar(x, binom.pmf(x, 10, 0.4))
axes[1].set_title("Binomial")

x = np.arange(15)
axes[2].bar(x, poisson.pmf(x, 4))
axes[2].set_title("Poisson")
x = np.linspace(-0.2, 1.2, 200)
axes[3].plot(x, uniform.pdf(x))
axes[3].set_title("Uniform")
x = np.linspace(-4, 4, 200)
axes[4].plot(x, norm.pdf(x))
axes[4].set_title("Gaussian")
plt.tight_layout()
plt.show()
prev = 0.01
sensitivity = 0.95
specificity = 0.90
posterior = (
   sensitivity * prev /
   (sensitivity * prev + (1 - specificity) * (1 - prev))
```

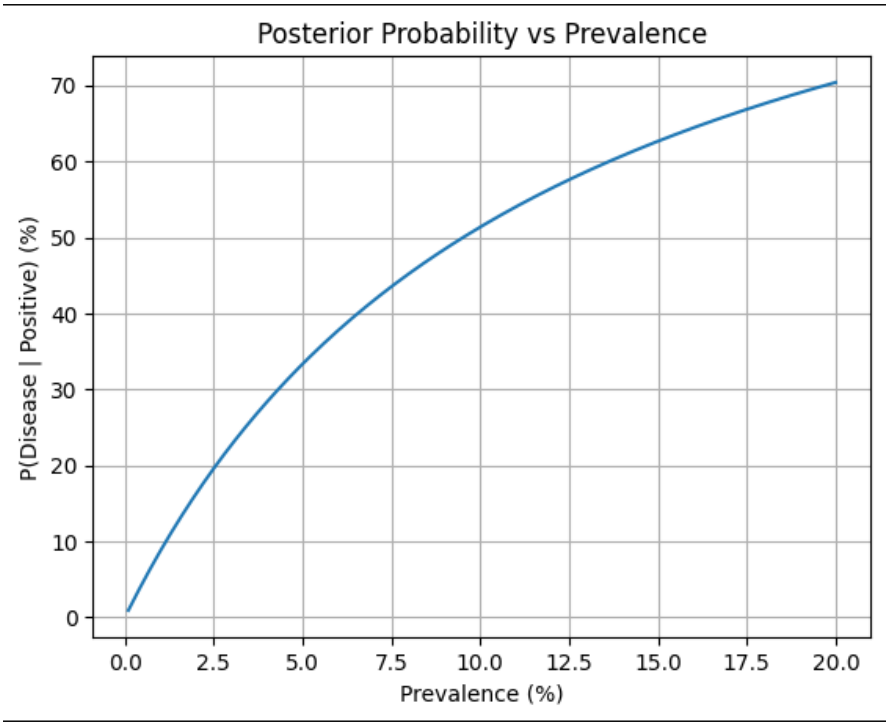
```
)
print("P(Disease | Positive):", round(posterior, 4))
print("Percentage:", round(posterior * 100, 2), "%")
N = 100_000
disease = np.random.random(N) < prev
positive = np.where(
    disease,
    np.random.random(N) < sensitivity,
    np.random.random(N) < (1 - specificity)
)
empirical = np.sum(disease & positive) / np.sum(positive)
print("Positive tests:", np.sum(positive))
print("True positive tests:", np.sum(disease & positive))
print("Empirical posterior:", round(empirical, 4))
p = np.linspace(0.001, 0.20, 200)
post = sensitivity * p / (
    sensitivity * p + (1 - specificity) * (1 - p)
)
plt.plot(p * 100, post * 100)
plt.xlabel("Prevalence (%)")
plt.ylabel("P(Disease | Positive) (%)")
plt.title("Posterior Probability vs Prevalence")
plt.grid()
plt.show()
```

Output




```
P(Disease | Positive): 0.0876
```

```
Percentage: 8.76 %
Positive tests: 10923
True positive tests: 950
Empirical posterior: 0.087
```



Experiment 4: Entropy, Cross-Entropy and KL Divergence

Aim

To calculate entropy, cross-entropy and Kullback–Leibler (KL) divergence for weather distributions, verify their mathematical relationship, demonstrate KL divergence asymmetry, and plot binary cross-entropy loss.

Formulas

1. Entropy

$H(P) = -\Sigma [p_i \times \log_2(p_i)]$

2. Cross-Entropy

$H(P, Q) = -\Sigma [p_i \times \log_2(q_i)]$

3. KL Divergence

$D_{kl}(P \parallel Q) = \Sigma [p_i \times \log_2(p_i / q_i)]$

4. Relationship Between Entropy, Cross-Entropy and KL Divergence

Cross-Entropy = Entropy + KL Divergence

$H(P, Q) = H(P) + D_{kl}(P \parallel Q)$

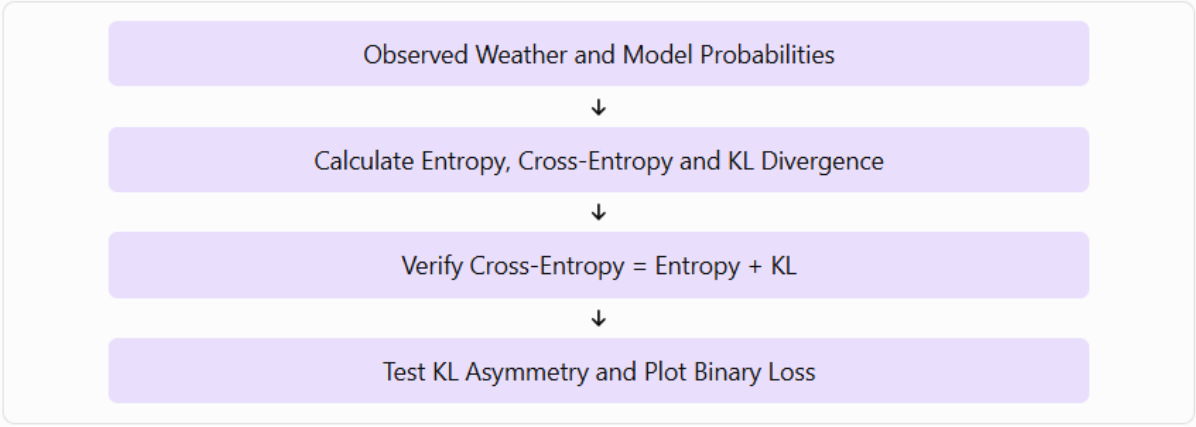
5. Reverse KL Divergence

$D_{kl}(Q \parallel P) = \Sigma [q_i \times \log_2(q_i / p_i)]$

6. Binary Cross-Entropy Loss

$L = -[y \times \log(p) + (1 - y) \times \log(1 - p)]$

Architecture diagram



Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import entropy
true = np.array([15, 10, 5]) / 30
model1 = np.array([0.50, 0.30, 0.20])
model2 = np.array([0.60, 0.25, 0.15])
H = entropy(true)
def evaluate_model(P, Q):
    CE = -np.sum(P * np.log(Q))
    KL = entropy(P, Q)
    return CE, KL
ce1, kl1 = evaluate_model(true, model1)
ce2, kl2 = evaluate_model(true, model2)
print("True entropy =", round(H, 4))
print("\nModel 1: CE =", round(ce1, 4),
      "KL =", round(kl1, 4))
print("Model 1: H + KL =", round(H + kl1, 4))
print("\nModel 2: CE =", round(ce2, 4),
      "KL =", round(kl2, 4))
print("Model 2: H + KL =", round(H + kl2, 4))
print("\nIdentity verified for Model 1:",
```

```
    np.isclose(ce1, H + kl1))
print("Identity verified for Model 2:",
    np.isclose(ce2, H + kl2))
kl_pq = entropy(true, model1)
kl_qp = entropy(model1, true)
print("\nKL(True || Model 1) =", round(kl_pq, 4))
print("KL(Model 1 || True) =", round(kl_qp, 4))
y_true = 1
pred = np.linspace(0.001, 0.999, 300)
loss_positive = -np.log(pred)
loss_negative = -np.log(1 - pred)
plt.plot(pred, loss_positive, label="Actual class = 1")
plt.plot(pred, loss_negative, label="Actual class = 0")
plt.xlabel("Predicted Probability of Class 1")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Binary Cross-Entropy Loss")
plt.legend()
plt.grid(True)
plt.show()
```

Output

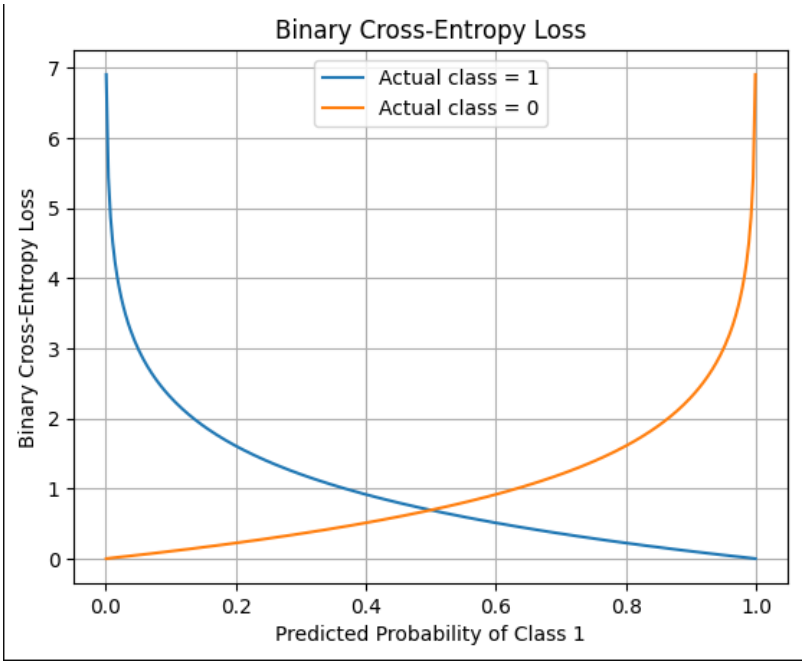
```
True entropy = 1.0114

Model 1: CE = 1.0161 KL = 0.0047
Model 1: H + KL = 1.0161

Model 2: CE = 1.0337 KL = 0.0223
Model 2: H + KL = 1.0337

Identity verified for Model 1: True
Identity verified for Model 2: True

KL(True || Model 1) = 0.0047
KL(Model 1 || True) = 0.0049
```



Experiment 5: Bias–Variance, Overfitting and Underfitting Analysis

Aim

To fit polynomial regression models of degrees 1, 3, 9 and 15, compare their fitted curves, plot training and validation errors, estimate bias squared and variance using repeated resampling, and select the best model.

Formulas

1. Polynomial Regression

$y = b_0 + b_1x + b_2x^2 + \ldots + b_nx^n$

2. Mean Squared Error (MSE)

$MSE = \Sigma\ (y_i - \hat{y}_i)^2 / N$

3. Bias

$Bias = E[\ \hat{f}(x)\ ] - f(x)$

4. Bias Squared

$Bias^2 = [\ E[\ \hat{f}(x)\ ] - f(x)\ ]^2$

5. Variance

$Variance = E[\ (\ \hat{f}(x) - E[\ \hat{f}(x)\ ]\ )^2\ ]$

6. Bias–Variance Decomposition

$MSE = Bias^2 + Variance + Irreducible\ Error$

7. Training Error

$Training\ Error = \Sigma\ (y_i - \hat{y}_i)^2 / N_train$

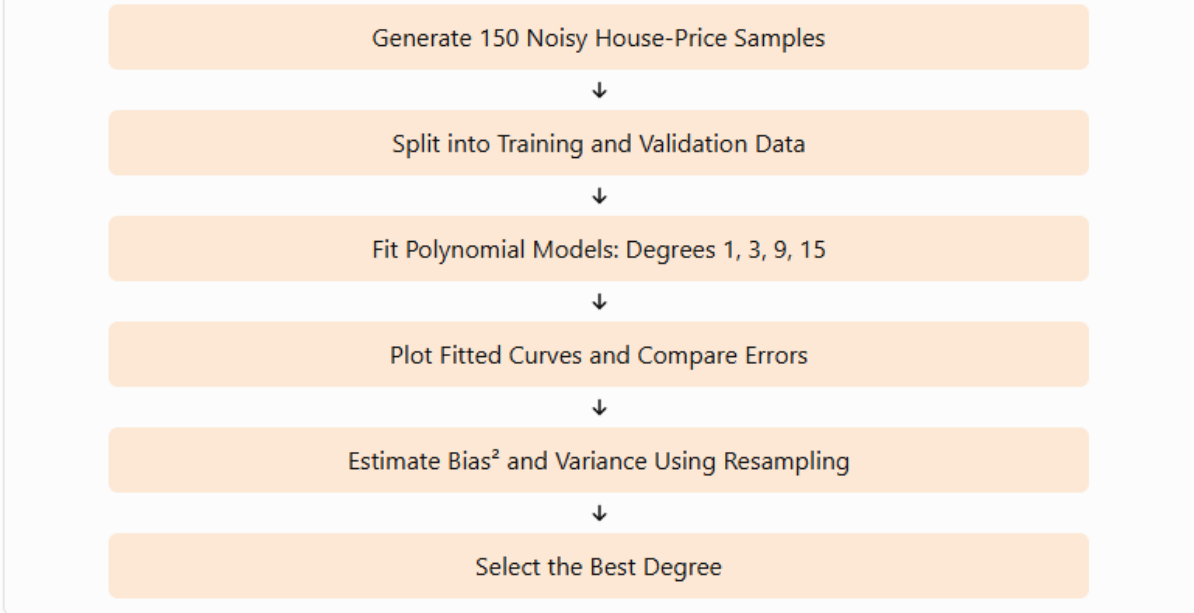
8. Validation Error

$Validation\ Error = \Sigma\ (y_i - \hat{y}_i)^2 / N_validation$

9. Best Model Selection

Best Degree = Degree with Minimum Validation Error

Architecture diagram



Code

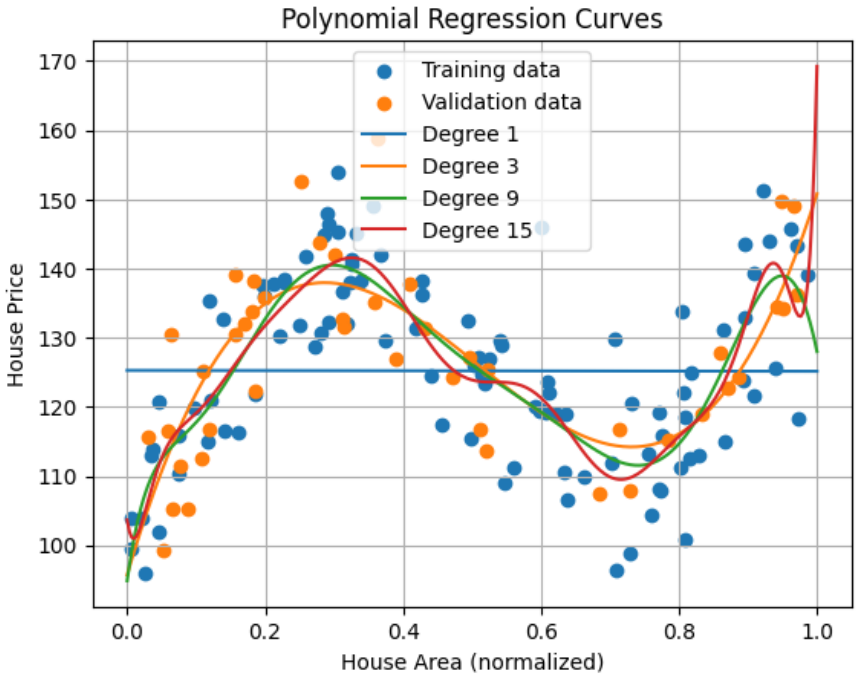
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error
np.random.seed(42)
X = np.sort(np.random.uniform(0, 1, 150)).reshape(-1, 1)
def true_function(x):
    return 100 + 50*x + 25*np.sin(2*np.pi*x)
y = true_function(X.ravel()) + np.random.normal(0, 8, 150)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.3, random_state=42
)
degrees = [1, 3, 9, 15]
models = {}
train_error = []
```

```
val_error = []
for d in degrees:
    model = make_pipeline(
        PolynomialFeatures(d),
        LinearRegression()
    )
    model.fit(X_train, y_train)
    models[d] = model
    train_error.append(mean_squared_error(
        y_train, model.predict(X_train)
    ))
    val_error.append(mean_squared_error(
        y_val, model.predict(X_val)
    ))
    print(f"Degree {d}: Training MSE = {train_error[-1]:.2f}, "
          f"Validation MSE = {val_error[-1]:.2f}")
grid = np.linspace(0, 1, 300).reshape(-1, 1)
plt.scatter(X_train, y_train, label="Training data")
plt.scatter(X_val, y_val, label="Validation data")
for d, model in models.items():
    plt.plot(grid, model.predict(grid), label=f"Degree {d}")
plt.xlabel("House Area (normalized)")
plt.ylabel("House Price")
plt.title("Polynomial Regression Curves")
plt.legend()
plt.grid()
plt.show()
plt.plot(degrees, train_error, marker="o", label="Training MSE")
plt.plot(degrees, val_error, marker="s", label="Validation MSE")
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
```

```
plt.title("Error vs Model Complexity")
plt.legend()
plt.grid()
plt.show()
rng = np.random.default_rng(42)
grid = np.linspace(0, 1, 100).reshape(-1, 1)
actual = true_function(grid.ravel())
bias2 = []
variance = []
for d in degrees:
    predictions = []

    for _ in range(100):
        indices = rng.choice(
            len(X_train), len(X_train), replace=True
        )
        model = make_pipeline(
            PolynomialFeatures(d),
            LinearRegression()
        )
        model.fit(X_train[indices], y_train[indices])
        predictions.append(model.predict(grid))
    predictions = np.array(predictions)
    mean_prediction = predictions.mean(axis=0)
    b2 = np.mean((mean_prediction - actual)**2)
    var = np.mean(np.var(predictions, axis=0))
    bias2.append(b2)
    variance.append(var)
    print(f"Degree {d}: Bias^2 = {b2:.3f}, Variance = {var:.3f}")
best = degrees[np.argmin(val_error)]
print("\nBest degree:", best)
```

print("Minimum validation MSE:", round(min(val_error), 2))



plt.plot(degrees, bias2, marker="o", label="Bias squared")

plt.plot(degrees, variance, marker="s", label="Variance")

plt.xlabel("Polynomial Degree")

plt.ylabel("Estimated Error")

plt.title("Bias-Variance Analysis")

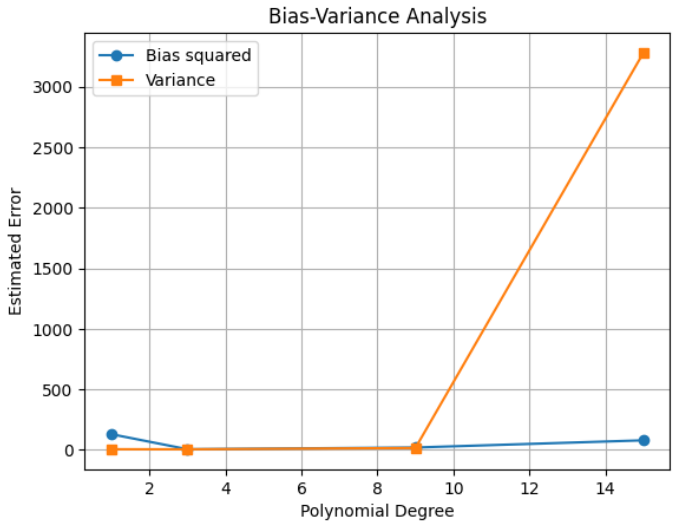
plt.legend()

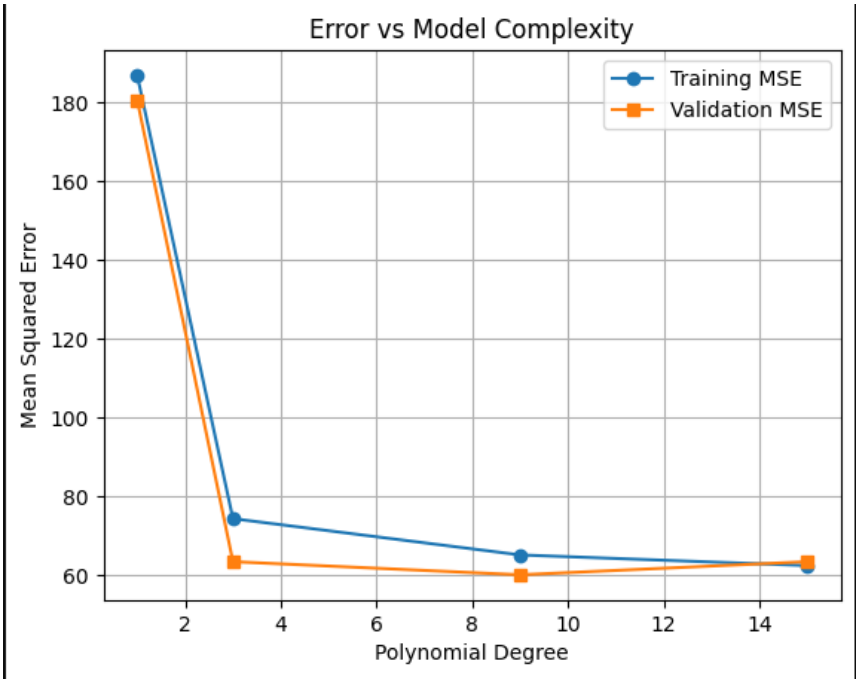
plt.grid()

plt.show()

Output

```
Degree 1: Training MSE = 186.84, Validation MSE = 180.57
Degree 3: Training MSE = 74.40, Validation MSE = 63.49
Degree 9: Training MSE = 65.21, Validation MSE = 60.16
Degree 15: Training MSE = 62.51, Validation MSE = 63.49
```






```
Degree 1: Bias^2 = 128.621, Variance = 4.498
Degree 3: Bias^2 = 4.309, Variance = 3.889
Degree 9: Bias^2 = 18.525, Variance = 14.478
Degree 15: Bias^2 = 77.979, Variance = 3281.473

Best degree: 9
Minimum validation MSE: 60.16
```